ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

MERGING BERT EXPERTS WITH WEIGHT-ENSEMBLING MOE

Few-shot router training across five GLUE tasks

Group 1
Nguyễn Bá Anh – 20261218M
Nguyễn Thị Nhã Linh – 20261262M
Instructor: Nguyễn Kiêm Hiếu

ONE LOVE. ONE FUTURE.

Motivation: one model, many task experts

Five fine-tuned experts
score **78.2** 547.4M params
strong, but five checkpoints

Task Arithmetic (static)
score **57.4** 109.5M params
compact, but loses performance

WEMoE (input-conditioned)
score **?** storage **?**
one merged model

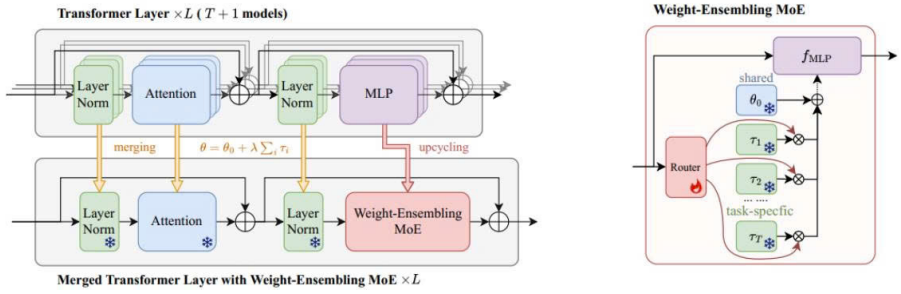
Question: can input-conditioned MLP weights recover task performance with one merged model, at lower storage than five experts?

Scope

BERT/GLUE adaptation of WEMoE, not a reproduction of the paper's CLIP/image experiments.[1]

[1] Anke Tang et al. "Merging Multi-Task Models via Weight-Ensembling..." (2024).

Theory: route in task-vector space



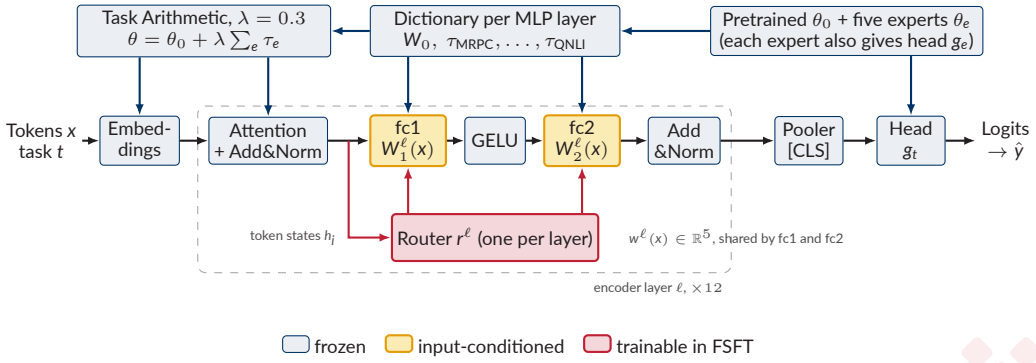
Left: merge non-MLP blocks, upcycle MLPs. Right: WEMoE module. Figure from Tang et al. (CLIP-ViT, pre-norm layer).[1]

Task vector	Non-MLP blocks (static)	MLP blocks (per input)
$\tau_e = W_e - W_0$	$\theta = \theta_0 + \lambda \sum_e \tau_e$	$W(x) = W_0 + \sum_{e=1}^{E} w_e(x)\, \tau_e$

- W_0: shared pretrained knowledge; τ_e: task-specific change, the "expert".
- $w_e(x)$ are **unnormalised**: they may amplify or subtract task vectors.

[1] Anke Tang et al. "Merging Multi-Task Models via Weight-Ensembling..." (2024).

Method: BERT MLPs become input-conditioned



- **Full** scope: fc1 + fc2 (`intermediate.dense`, `output.dense`). **Intermediate-only**: fc1; fc2 uses Task Arithmetic.
- All other weights use Task Arithmetic. Experts do **not** run at inference: one forward pass; the task id selects the head.

Components: frozen experts, trainable routers

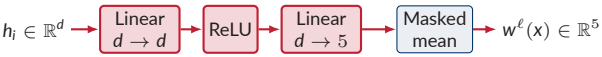
Expert e: full fine-tune, then frozen



$$f_e(x) = g_e\big(\mathrm{BERT}(x;\theta_e)\big)$$

Output to WEMoE: task vectors τ_e and head g_e.
Tasks: MRPC, RTE, CoLA, SST-2, QNLI.

Router r^ℓ: one per layer, 7.1M parameters in total



$$r(h) = A_2\,\mathrm{ReLU}(A_1 h + b_1) + b_2$$

$$w^\ell(x) = \frac{\sum_i m_i\, r^\ell(h_i)}{\sum_i m_i} \quad (m_i\text{: non-padding mask})$$

- **No softmax, no top-k:** $w^\ell(x)$ is a dense real-valued vector, not a probability distribution.
- **Initialisation:** $b_2 = \lambda$, small A_1, A_2, so $w_e(x) \approx \lambda$ and the model starts near Task Arithmetic.
- **Per-sample merge:** the same $w^\ell(x)$ mixes weights and biases of the layer's upscaled modules.

Pipeline: what each stage optimises or computes

Stage	Phase	Equation	Updates
1. Fine-tune experts	Training	$\min_{\theta_e,\, g_e} \mathbb{E}_{(x,y)\sim D_e}\big[-\log p(y \mid x;\theta_e,g_e)\big], \quad p=\mathrm{softmax}\, f_e(x)$	One expert
2. Merge	Build	$\tau_e=\theta_e-\theta_0; \quad \theta_{\text{non-MLP}}=\theta_0+\lambda\sum_e \tau_e$	None
3. FSFT	Training	$\min_{\phi} \sum_{t=1}^{5} \frac{1}{\lvert B_t\rvert} \sum_{(x,y)\in B_t} -\log p_\phi(y \mid x,t)$	Routers ϕ only
4. Predict	Inference	$W^\ell(x)=W_0^\ell+\sum_e w_e^\ell(x)\,\tau_e^\ell; \quad \hat{y}=\arg\max_c \ \mathrm{softmax}\big(g_t(\mathrm{BERT}_{W(x)}(x))\big)_c$	None

- B_t: batch of 16 from k few-shot examples of task t. AdamW, lr 10^{-4}, 10% warmup, 500 steps.
- The paper trains routers by unlabelled entropy minimisation (TTA); this project uses labelled cross-entropy.[1]

HUST

[1] Anke Tang et al. "Merging Multi-Task Models via Weight-Ensembling..." (2024).

Data boundary: reserve router examples, select on dev

Portion of each official GLUE train split	Size	Used for
`expert_train`	Remaining examples	Fine-tune the task experts
`fewshot_pool`	256 per task	FSFT router training only
`dev`	10%, at most 1,000	Choose expert epoch, λ, and router step
Official validation	Task-specific	Final metrics only

Leakage control

Experts never see the few-shot pool or dev; official validation is never used for selection.

Experiments: compare merging, routing, and expert limits

Model	Merge rule	Selection / training
Majority class	Constant prediction	None
Simple Averaging	$\theta_0 + \frac{1}{5}\sum_e \tau_e$	None
Task Arithmetic	$\theta_0 + \lambda \sum_e \tau_e$	λ on dev (best 0.3)
WEMoE init	$w_e(x) \approx \lambda$	Router initialisation only
WEMoE + FSFT	$W_0 + \sum_e w_e(x)\tau_e$	CE on k per task; dev every 50 steps, up to 500
Five experts	No merge	Best epoch on dev

Task score ($\times 100$)

MRPC: mean of accuracy and F1; CoLA: Matthews correlation (MCC); RTE, SST-2, QNLI: accuracy.

Task Arithmetic: Ilharco et al.[1]

HUST

[1] Gabriel Ilharco et al. "Editing Models with Task Arithmetic" (2023).

Main result: FSFT recovers much of the merging gap

Model	Average task score
Majority class	45.8
Simple Averaging ($\lambda = 1/5$)	53.3
Task Arithmetic ($\lambda = 0.3$)	57.4
WEMoE init (before FSFT)	57.8 ± 0.6
WEMoE + FSFT, full MLP ($k = 256$)	$\mathbf{69.9 \pm 0.1}$
Five single-task experts	78.2

- Gain over majority $= \frac{\text{score} - \text{majority}}{\text{expert} - \text{majority}}$: Task Arithmetic **18%**, WEMoE + FSFT **59%**.
- Router initialisation alone does not help (57.8): the gain comes from FSFT.
- Mean $\pm$ std over router seeds 0, 1, 2.

Few-shot budget: $k = 256$ is stable, small k is not

k per task	Avg score	Gain over majority	CoLA MCC
16	60.8 ± 2.7	25.5%	27.6 ± 14.1
64	64.6 ± 3.7	46.7%	20.5 ± 17.8
256	$\mathbf{69.9 \pm 0.1}$	59.1%	42.6 ± 0.9

- Full-MLP WEMoE; mean $\pm$ std over router seeds 0, 1, 2.
- Small k overfits early (best dev step 50–250 of 500) and varies across seeds.
- $k = 16 \rightarrow 64$: average +3.8, gain almost doubles; part of the $k = 16$ average is MRPC/RTE at constant-guess level.

Per-task results: aggregate gains hide a task trade-off

	MRPC acc/F1	RTE acc	CoLA MCC	SST-2 acc	QNLI acc
Majority	68.4/81.2	52.7	0.0	50.9	50.5
Full MLP, $k = 256$	76.3 ± 4.5 / 83.9 ± 2.8	52.0 ± 4.4	42.6 ± 0.9	88.0 ± 0.5	86.7 ± 0.7
Intermediate, $k = 256$	66.5 ± 0.7 / 76.6 ± 1.0	59.0 ± 0.9	41.7 ± 0.9	86.7 ± 0.2	86.5 ± 0.2
Five experts	82.1/88.1	63.2	59.1	92.2	91.2

Do not read the mean as universal dominance

Full MLP is below the RTE majority (52.0 vs. 52.7). Intermediate-only improves RTE but is below the MRPC majority (66.5/76.6 vs. 68.4/81.2).

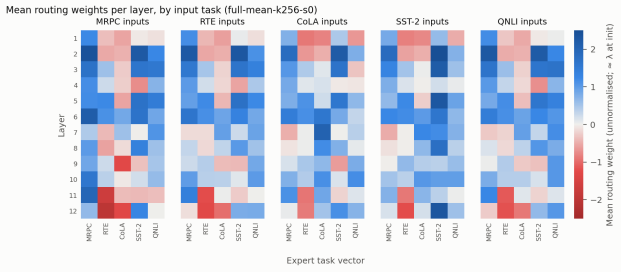
RTE ($n = 277$) and MRPC ($n = 408$): standard error about 3 and 2 points, so part of the gap may be noise.

Storage trade-off: intermediate-only is smaller, not uniformly better

Deployment	Stored params	vs. five models	Avg score	Task contrast
Five experts	547.4M	100%	78.2	Strongest average
WEMoE, full MLP	400.0M	73.1%	69.9 ± 0.1	MRPC 76.3; RTE 52.0
WEMoE, intermediate	258.4M	47.2%	69.1 ± 0.1	MRPC 66.5; RTE 59.0

- Intermediate-only: **35% fewer parameters** than full MLP, similar average; best step is late (400–500 of 500).
- **Caveat:** each task keeps its own head, so the task is known at inference. Five experts are then more accurate and only 37% larger than full MLP.
- The storage case rests on intermediate-only, or on settings with unknown task.
- FSFT on one Tesla T4: 14–16 min (full), 8.5 min (intermediate); training time, not inference latency.

Routing: shared structure, and why full-MLP fails on RTE



- **Shared pattern:** mean-matrix correlation 0.82; input task explains about 58% of variance; weights span -3.0 to 4.4.
- **RTE, full MLP:** own expert weight ≈ 0.01 (MRPC: 1.37), and -1.2 in layers 11–12. Intermediate-only: 0.96, RTE 59.0.

Full MLP, $k = 256$, seed 0. Input-independent ablation pending: this plot does not show that input dependence causes the gains.

Limitations: encouraging aggregate, unresolved failures

- **RTE failure:** full MLP 52.0, below the 52.7 majority; the RTE expert is also weak (63.2).
- **Small budgets vary:** at $k = 16$, CoLA MCC std is 14.1 across router seeds.
- **Selection sees more labels:** up to 1,000 dev examples per task vs. $k = 256$ for FSFT.
- **Limited seeds:** one seed for experts and data split.

Paper: CLIP/image tasks, unlabelled test-time entropy minimisation. Here: 256 labelled examples per task. Comparison is qualitative.[1]

[1] Anke Tang et al. "Merging Multi-Task Models via Weight-Ensembling..." (2024).

Conclusion: WEMoE helps, but the task-level trade-offs matter

Answer

In this first full BERT/GLUE run, few-shot router training raises the average to 69.9 (Task Arithmetic: 57.4; experts: 78.2).

- Full MLP and intermediate-only: similar averages, different storage and per-task outcomes.
- RTE stays below majority for full MLP; an aggregate does not imply per-task dominance.
- Next: more seeds, stricter few-shot selection, routing ablation, inference latency.

Backup: Transformer architecture

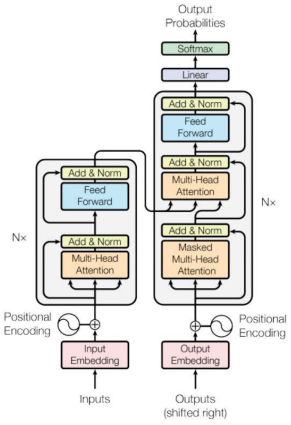


Figure: Vaswani et al.[1]

Encoder: self-attention + FFN. **Decoder:** adds masked self-attention and cross-attention.

Self-attention

$$\text{Attention}(Q, K, V) = \text{softmax}\Big(\frac{QK^\top}{\sqrt{d_k}}\Big)V$$

Multi-head attention

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O$$

Positional encoding: sinusoidal vectors added to the embeddings, because attention does not encode order.

FFN and residual

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2, \quad y = \text{LayerNorm}\big(x + \text{Sublayer}(x)\big)$$

[1] Ashish Vaswani et al. "Attention Is All You Need" (2017).

Backup: BERT architecture

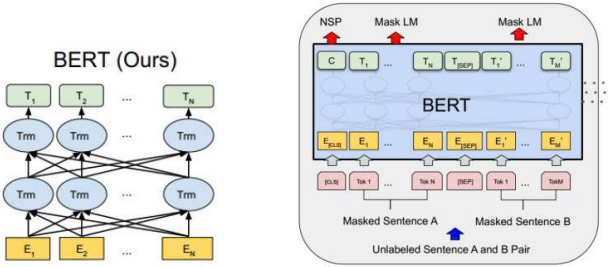


Figure: Devlin et al. (encoder stack; MLM and NSP pre-training).[1]

- Encoder-only, bidirectional context.
- Input: `[CLS]` A `[SEP]` B `[SEP]`.
- $E = E_{\text{token}} + E_{\text{segment}} + E_{\text{position}}$ (learned positions).

- Pre-training: MLM (predict masked tokens), NSP.
- `bert-base-uncased`: 12 layers, GELU.
- This project starts from the encoder and pooler only, with new 2-class heads.

[1] Jacob Devlin et al. "BERT: Pre-training of Deep Bidirectional..." (2019).

Backup: BERT fine-tuning for GLUE tasks

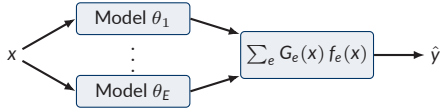
Task	Input	Label	Task score	Expert score
MRPC	Sentence pair	not equivalent / equivalent	mean(acc, F1)	85.1
RTE	Sentence pair	entailment / not entailment	Accuracy	63.2
CoLA	One sentence	unacceptable / acceptable	MCC	59.1
SST-2	One sentence	negative / positive	Accuracy	92.2
QNLI	Question + sentence	entailment / not entailment	Accuracy	91.2

	Pretrained BERT, θ_0	Fine-tuned expert, θ_e
Objective	Self-supervised MLM + NSP	Cross-entropy on one GLUE task
Head	None kept	Dropout + Linear ($d \to 2$) on pooled `[CLS]`
Updated	–	All BERT layers and the head
Role in WEMoE	Shared base W_0, θ_0	Task vector $\tau_e = \theta_e - \theta_0$, head g_e

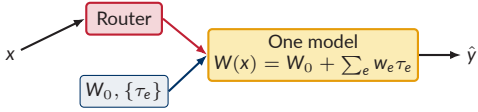
- One recipe for all tasks: AdamW, lr 2×10^{-5}, batch 32, 3 epochs, 10% warmup, weight decay 0.01, length 128.
- All five tasks are binary classification, so no regression head is used.

Backup: weight ensembling vs. logit ensembling

Logit ensembling (concept only)



Weight ensembling (WEMoE)



	Logit ensembling	Weight ensembling
Mixes	Model outputs	Parameters, inside each upscaled MLP
Forward passes	One per expert	One, with per-sample weights
Stored	Full models	Shared model + MLP task vectors (283M of 400M)
Limitation	Cost grows with E; needs a shared output space	Storage for τ_e; per-sample merge cost

For one linear layer, $W(x)h = W_0 h + \sum_e w_e(\tau_e h)$. GELU and 12 stacked layers make the full model differ from a logit ensemble.

Backup: router and Mixture of Experts

$$\underbrace{\mathrm{MoE}(x) = \sum_{i=1}^{n} G_i(x)\, E_i(x)}_{\text{classic MoE: mixes expert outputs}} \quad \longrightarrow \quad \underbrace{h_{\text{out}} = f_{\text{MLP}}\big(h;\ W_0 + D_\tau w(x)\big)}_{\text{WEMoE: mixes expert weights}}$$

	Classic MoE	This project
Experts	Trainable sub-networks	Frozen task vectors τ_e (D_τ: dictionary)
Gate output	Weight distribution; may keep top-k	Unnormalised $w \in \mathbb{R}^5$; no softmax
Routing	Often sparse: few experts active	**Dense**: all 5 task vectors in every layer
Gate placement	Per MoE layer	One router per BERT layer; fc1, fc2 share w
Training	Jointly with experts	Routers only, after experts are frozen

Negative weights are allowed: routing here is not a convex mixture of experts. Classic MoE as in Shazeer et al.[1]

[1] Noam Shazeer et al. "Outrageously Large Neural Networks: The..." (2017).

Backup: task metrics and aggregation

- **MRPC:** mean of accuracy and F1. **CoLA:** MCC. **RTE, SST-2, QNLI:** accuracy.
- **Average score:** unweighted mean of five task scores, $\times 100$.
- **Uncertainty:** WEMoE mean $\pm$ std over router seeds 0, 1, 2; experts and data split fixed.

Backup: router training details

- 256 labelled examples per task are reserved for FSFT; experts never see them.
- Router: two linear layers with ReLU, applied per token; outputs are averaged over non-padding tokens.
- AdamW, lr 10^{-4}, batch 16 per task, up to 500 steps; 7.1M trainable parameters.
- Best step: dev every 50 steps, up to 1,000 examples per task.
- First full run: not a general guarantee or a complete reproduction of WEMoE.

Backup: WEMoE paper and this project

	Tang et al.	This project
Backbone, tasks	CLIP-ViT, 8 image tasks	BERT-base, 5 GLUE tasks
Upscaled modules	Transformer MLPs	Full MLP; intermediate-only as ablation
Routing input	Mean over tokens	Mean over non-padding tokens
Router training	Entropy on unlabelled test data	Cross-entropy on k labelled examples
Router output	Unnormalised	Unnormalised, bias init λ
Other parameters	Task Arithmetic, $\lambda = 0.3$	Task Arithmetic, λ chosen on dev

The paper has no GLUE results, so the comparison is qualitative.[1]

[1] Anke Tang et al. "Merging Multi-Task Models via Weight-Ensembling..." (2024).

References

[1] Anke Tang et al. *Merging Multi-Task Models via Weight-Ensembling Mixture of Experts*. 2024. arXiv: 2402.00433 [cs.LG].

[2] Gabriel Ilharco et al. *Editing Models with Task Arithmetic*. 2023. arXiv: 2212.04089 [cs.LG].

[3] Ashish Vaswani et al. *Attention Is All You Need*. 2017. arXiv: 1706.03762 [cs.CL].

[4] Jacob Devlin et al. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. 2019. arXiv: 1810.04805 [cs.CL].

[5] Noam Shazeer et al. *Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer*. 2017. arXiv: 1701.06538 [cs.LG].